Q2 – Tuples, Mutability and Copying

Tuples and dictionaries

A tuple is immutable and hashable when all its members are hashable, so a (latitude, longitude) tuple can be a dictionary key. A list is mutable and unhashable, so it cannot be a dictionary key. Tuple unpacking makes loops readable.

Aliasing and copying

The expression [[0]*cols]*rows repeats references to the same inner list. Changing one row changes every row. A nested list comprehension creates independent row objects. A shallow copy copies only the outer container; nested objects remain shared. deepcopy recursively copies nested objects.

Function arguments and measurements

A function receives a reference to the same mutable list object, so mutations are visible to the caller. sys.getsizeof reports only the container's shallow size, not all referenced elements. Compare list and tuple sizes on the same interpreter and data. A tuple containing a list can still have that list mutated; tuple immutability protects the tuple's references, not nested objects.

Alternative sparse grid

A dictionary keyed by (row, col) stores only occupied cells. Both list indexing and average dictionary lookup are O(1). A dense grid uses O(rows×cols) space; a sparse map uses O(non-empty cells), but absent cells need a default value.

Python Program

```
import copy
rows, cols = 3, 4
bad = [[0] * cols] * rows
bad[0][1] = 7
good = [[0 for _ in range(cols)] for _ in range(rows)]
good[0][1] = 7
print("bad :", bad)
print("good:", good)
print(bad[0] is bad[1], good[0] is good[1])
stops = {(13.0827, 80.2707): "Chennai Central", (13.0418, 80.2341): "T. Nagar"}
print(stops[(13.0418, 80.2341)])
try:
    stops[[13.0418, 80.2341]] = "invalid key"
except TypeError as exc:
    print(type(exc).__name__, exc)
nested = [[1], [2]]
shallow = copy.copy(nested)
deep = copy.deepcopy(nested)
nested[0].append(9)
print("shallow sees nested mutation:", shallow)
print("deep is independent:", deep)
```